MyoWare IMU v2 — Host Protocol & Recording System

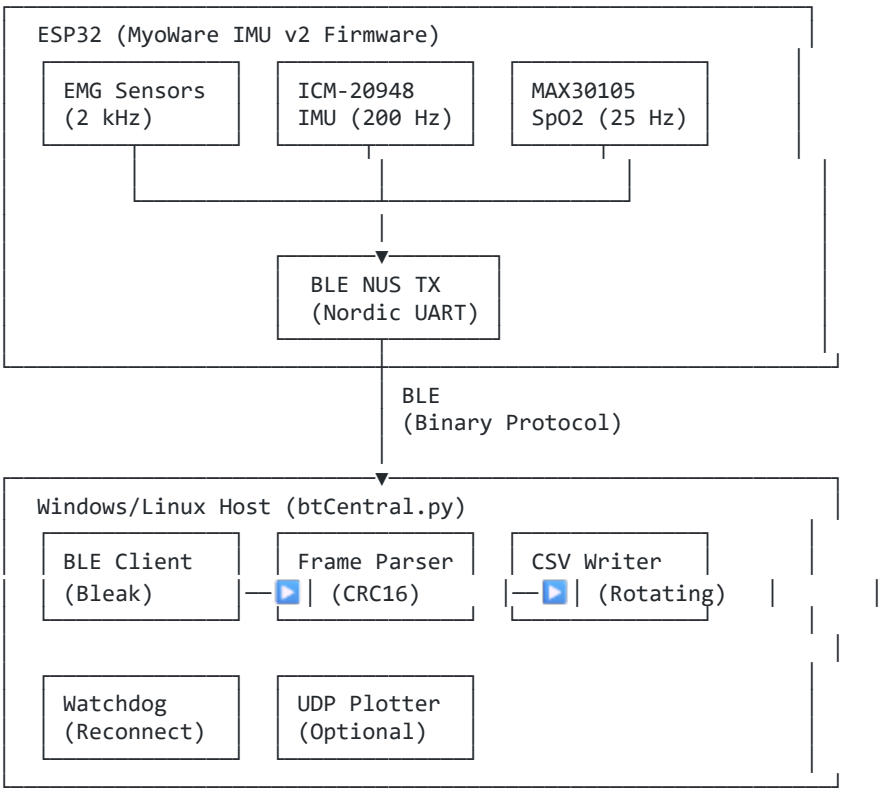
This document describes the BLE protocol, host-side recording architecture, and file management strategy for the MyoWare IMU v2 system.

Table of Contents

1. System Overview
2. BLE Protocol Specification
3. Host Recording Architecture
4. File Management Strategy
5. UDP Telemetry (Real-Time Plotting)
6. Watchdog & Reconnection
7. Data Flow Diagram

System Overview

Hardware Architecture



Key Features

- **High-throughput streaming:** 2 kHz EMG + 200 Hz IMU over BLE

- **Robust protocol:** CRC16 validation, frame sequencing, gap detection
- **Time-rotating files:** Split long sessions into manageable chunks (default 30 min)
- **Watchdog:** Auto-reconnect on BLE stalls (no data loss)
- **Real-time plotting:** Optional UDP telemetry for live visualization
- **Battery monitoring:** Track device battery level during recording

---

BLE Protocol Specification

Nordic UART Service (NUS)

UUID	Role	Description
`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`	Service	Nordic UART Service
`6E400003-B5A3-F393-E0A9-E50E24DCCA9E`	TX (notify)	Device → Host data stream
`6E400002-B5A3-F393-E0A9-E50E24DCCA9E`	RX (write)	Host → Device commands

Host → Device Commands (RX Characteristic)

Command	Format	Description
`PING`	ASCII string	Heartbeat to keep device streaming (sent every 1 s)
`ACK:<seq>`	ASCII string	Acknowledge frame sequence (optional)

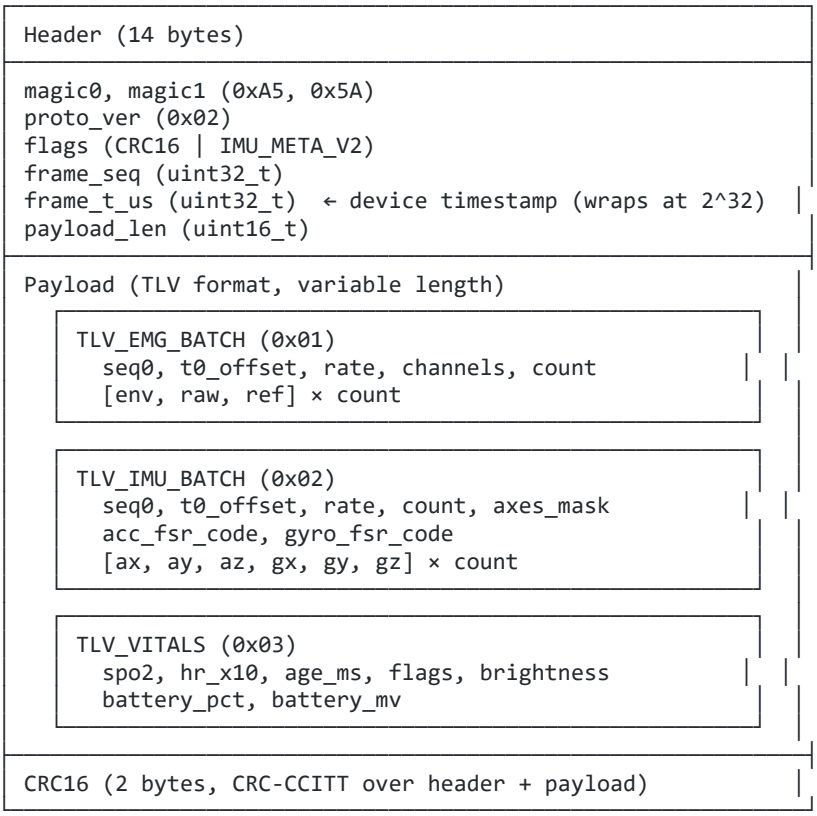
Watchdog Policy:

- Device expects `PING` every ~1 second
- Device pauses sampling after 5 seconds of silence
- Device reboots after 30 seconds of silence (recovers stuck BLE state)

Device → Host Data Stream (TX Characteristic)

Binary protocol with TLV (Type-Length-Value) payloads.

Frame Structure



Header Fields

Field	Type	Description
`magic0, magic1`	`uint8_t × 2`	Frame sync bytes: `0xA5, 0x5A`
`proto_ver`	`uint8_t`	Protocol version (current: `0x02`)
`flags`	`uint8_t`	Bit 0: CRC16 present, Bit 1: IMU meta v2
`frame_seq`	`uint32_t`	Frame sequence number (wraps at 2^{32})
`frame_t_us`	`uint32_t`	Device timestamp in microseconds (wraps at 2^{32})
`payload_len`	`uint16_t`	Length of TLV payload in bytes

Device Timestamp Wrapping:

- `frame_t_us` is a 32-bit counter that wraps every ~71.6 minutes
- Host extends to 64-bit by detecting wraps: `if (new < old && (old - new) > 0x40000000) wraps++`
- Extended timestamp: `(wraps << 32) + frame_t_us`

TLV Payload Format

Each TLV entry:

type (1)	len (2)	value (len bytes)

TLV*EMG*BATCH (0x01)

Meta (13 bytes):

```
struct EmgMeta {
    uint32_t seq0;        // First sample sequence number
    int16_t  t0_offset;   // Timestamp offset from frame_t_us (µs)
    uint16_t rate_hz;     // Sample rate (e.g., 2000 Hz)
    uint8_t  channels;    // Number of channels (2 or 3)
    uint8_t  mask;        // Channel mask (0x03 = ENV+RAW)
    uint8_t  reserved;
    uint16_t count;       // Number of samples in this batch
};
```

Samples (4 bytes per sample for 2 channels):

```
struct EmgSample {
    uint16_t env;  // Envelope (rectified, low-pass filtered)
    uint16_t raw;  // Raw EMG (rectified)
    uint16_t ref;  // Reference (optional, 3rd channel)
};
```

Timestamp Reconstruction:

```
t0_us = frame_t_us + t0_offset
for i in range(count):
    sample_t_us = t0_us + i * (1_000_000 // rate_hz)
```

Why Timestamp Offset?

- Saves 4 bytes per sample (vs. absolute timestamp)
- Device back-calculates actual ADC capture time (corrects ~16 ms DMA latency)
- Host reconstructs absolute time using frame timestamp + offset

TLV*IMU*BATCH (0x02)

Meta v2 (14 bytes):

```
struct ImuMeta {
    uint32_t seq0;        // First sample sequence number
    int16_t  t0_offset;   // Timestamp offset from frame_t_us (µs)
    uint16_t rate_hz;     // Sample rate (e.g., 200 Hz)
    uint8_t  count;       // Number of samples in this batch
    uint8_t  reserved;
    uint16_t axes_mask;   // Bit mask: 0x003F = all 6 axes
    uint8_t  acc_fsr_code;   // Accelerometer FSR: 0=±2g, 1=±4g, 2=±8g, 3=±16g
    uint8_t  gyro_fsr_code;  // Gyroscope FSR: 0=±250dps, 1=±500dps, 2=±1000dps, 3=±2000dps
};
```

Samples (12 bytes per sample):

```
struct ImuSample {
    int16_t ax, ay, az;  // Accelerometer (raw LSB)
    int16_t gx, gy, gz;  // Gyroscope (raw LSB)
};
```

Scaling to Physical Units:

```
# Accelerometer (LSB/g)
acc_scale = {2: 16384.0, 4: 8192.0, 8: 4096.0, 16: 2048.0}[acc_fsr_g]
acc_g = raw_lsb / acc_scale

# Gyroscope (LSB/dps)
gyro_scale = {250: 131.0, 500: 65.5, 1000: 32.8, 2000: 16.4}[gyro_fsr_dps]
gyro_dps = raw_lsb / gyro_scale
```

Why Log FSR in Every Frame?

- Allows host to scale raw LSB values to physical units (g, dps)
- Supports dynamic FSR changes (future feature)
- Self-documenting: no need to remember device config

TLV_VITALS (0x03)

Payload (11 bytes):

```
struct Vitals {
    uint16_t spo2_u16;      // SpO2 (0-100) or 0xFFFF if invalid
    uint16_t hr_x10_u16;    // Heart rate × 10 (bpm) or 0xFFFF if invalid
    uint16_t age_ms;        // Age of SpO2/HR data (ms since computed)
    uint8_t  flags;         // Bit 0: SpO2 valid, Bit 1: HR valid
    uint8_t  brightness;    // MAX30105 LED brightness (0x0F-0x7F)
    uint8_t  battery_pct;   // Battery % (0-100) or 255 if unknown
    uint16_t battery_mv;    // Battery voltage (mV) or 0 if unknown
};
```

Backward Compatibility:

- Old firmware (8 bytes): No battery fields
- New firmware (11 bytes): Includes `battery_pct` and `battery_mv`
- Host parser handles both formats

Timestamp Reconstruction:

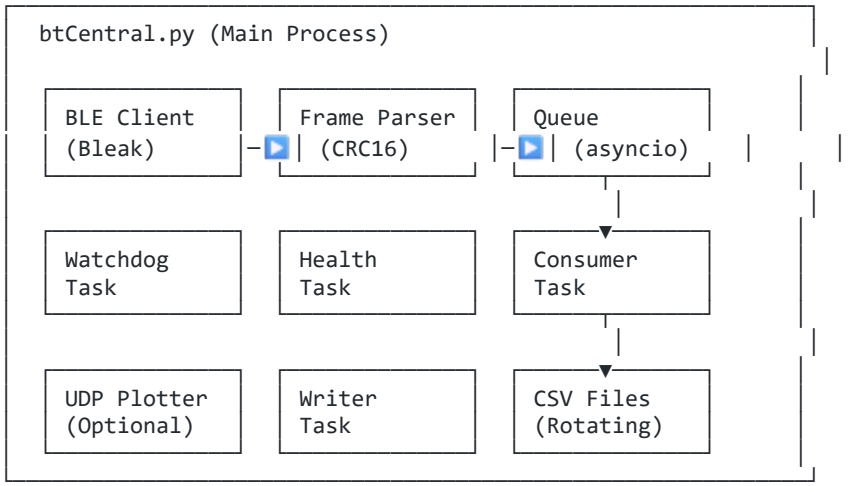
```
dev_computed_us = frame_t_us - (age_ms * 1000)
```

Why Age Field?

- SpO2/HR are computed every ~30 seconds (slow)
- Age tells host how stale the data is
- Allows host to interpolate or flag old data

Host Recording Architecture

Component Overview



Task Architecture

Task	Purpose	Rate
BLE Client	Receives BLE notifications, enqueues raw bytes	Event-driven
Consumer Task	Parses frames, decodes TLVs, buffers rows	~1 kHz (batch 64 chunks)
Writer Task	Flushes buffered rows to CSV files	Every 2 seconds
Health Task	Prints stats, checks battery, detects stalls	Every 1 second
Watchdog Task	Monitors last notification time, triggers reconnect	Every 0.5 seconds
Ping Task	Sends `PING` to device	Every 1 second

Data Flow

```
BLE Notify (Core 0)
    ↓
enqueue_chunk() → asyncio.Queue (max 20,000 chunks)
    ↓
Consumer Task (batch 64 chunks)
    ↓
FrameParser.feed() → CRC16 validation
    ↓
decode_emg() / decode_imu() / decode_vitals()
    ↓
Append to emg_rows / imu_rows / vitals_rows (in-memory buffers)
    ↓
Writer Task (every 2 s)
    ↓
CSV files (time-rotating)
```

Queue Management

Why a Queue?

- BLE notifications arrive on a separate thread (Bleak callback)
- Queue decouples BLE I/O from parsing/writing (prevents blocking)
- Allows batching: consumer drains up to 64 chunks per iteration

Queue Full Policy:

- Max size: 20,000 chunks (configurable via `--queue-max`)
- On overflow: drop oldest chunk, increment `queue_drops` counter
- Health report shows `q_drop` to detect sustained overload

Typical Queue Depth:

- Normal operation: 0-50 chunks
- BLE burst: 100-500 chunks (transient)
- Sustained >1000: indicates host CPU overload or slow disk I/O

File Management Strategy

Design Philosophy

Single Session = Single Folder

All files within one output directory (`--outdir logs`) belong to a **single tennis session**. Time-rotating files are used to:

1. **Prevent data loss:** If a file is corrupted, only one segment is lost (not the entire session)
2. **Enable streaming analysis:** Process completed segments while recording continues
3. **Manage file size:** Keep individual files <500 MB for easier handling

Rotation is NOT session boundaries — it's purely a file management strategy.

File Naming Convention

```
logs/
├── emg_log_<session_id>_<open_id>_<idx>.csv
├── emg_log_<session_id>_<open_id>_<idx>.meta.json
├── imu_log_<session_id>_<open_id>_<idx>.csv
├── imu_log_<session_id>_<open_id>_<idx>.meta.json
└── vitals_log.csv  (append-only, no rotation)
```

Filename Components:

Component	Format	Example	Description
`<session_id>`	`YYYYMMDD_HHMMSS`	`20250128_143022`	Session start time (when `btCentral.py` launched)
`<open_id>`	`YYYYMMDD_HHMMSS`	`20250128_143022`	File open time (first file = session start)
`<idx>`	`000`, `001`, ...	`000`	Rotation index (increments every `--rotate-min` minutes)

Example Session (60 min recording, 30 min rotation):

```
logs/
├── emg_log_20250128_143022_20250128_143022_000.csv       ← 0-30 min
├── emg_log_20250128_143022_20250128_143022_000.meta.json
├── emg_log_20250128_143022_20250128_143022_001.csv       ← 30-60 min
├── emg_log_20250128_143022_20250128_143022_001.meta.json
├── imu_log_20250128_143022_20250128_143022_000.csv
├── imu_log_20250128_143022_20250128_143022_000.meta.json
├── imu_log_20250128_143022_20250128_143022_001.csv
├── imu_log_20250128_143022_20250128_143022_001.meta.json
└── vitals_log.csv  (all vitals from all sessions, append-only)
```

Why `<open_id>` in Filename?

- Allows multiple recording sessions in the same folder (different `<session_id>`)
- Each session's files are grouped by `<session_id>` prefix
- `<open_id>` distinguishes rotation segments within a session

CSV File Formats

EMG Log (`emg_log_*.csv`)

Column	Type	Description
`dev_us`	`int64`	Device timestamp (µs, extended to 64-bit)
`emg_seq`	`uint32`	EMG sample sequence number
`env`	`uint16`	Envelope (rectified, low-pass filtered)
`raw`	`uint16`	Raw EMG (rectified)
`ref`	`uint16`	Reference channel (0 if not connected)
`frame_seq`	`uint32`	Frame sequence number
`host_rx_ms`	`int64`	Host receive timestamp (ms since epoch)

Sample Rate: 2000 Hz (typical)
File Size: ~30 MB per 30 min segment

IMU Log (`imu_log_*.csv`)

Column	Type	Description
`dev_us`	`int64`	Device timestamp (µs, extended to 64-bit)
`imu_seq`	`uint32`	IMU sample sequence number
`ax, ay, az`	`int16`	Accelerometer (raw LSB)
`gx, gy, gz`	`int16`	Gyroscope (raw LSB)
`acc_fsr_g`	`uint8`	Accelerometer FSR (2, 4, 8, or 16 g)
`gyro_fsr_dps`	`uint16`	Gyroscope FSR (250, 500, 1000, or 2000 dps)
`frame_seq`	`uint32`	Frame sequence number
`host_rx_ms`	`int64`	Host receive timestamp (ms since epoch)

Sample Rate: 200 Hz (typical)
File Size: ~3 MB per 30 min segment

Vitals Log (`vitals_log.csv`)

Column	Type	Description
`dev_us_frame`	`int64`	Device timestamp of frame (µs)
`dev_us_computed`	`int64`	Device timestamp when vitals computed (µs)
`spo2`	`float`	SpO2 (%) or empty if invalid
`hr_bpm`	`float`	Heart rate (bpm) or empty if invalid
`age_ms`	`uint16`	Age of vitals data (ms)
`flags`	`uint8`	Validity flags
`brightness`	`uint8`	MAX30105 LED brightness
`battery_pct`	`uint8`	Battery % (0-100) or empty if unknown
`battery_v`	`float`	Battery voltage (V) or empty if unknown
`frame_seq`	`uint32`	Frame sequence number
`host_rx_ms`	`int64`	Host receive timestamp (ms since epoch)

Sample Rate: ~1 sample per 30 seconds
File Size: <1 KB per hour (append-only, no rotation)

Metadata Files (`*.meta.json`)

Each CSV file has a corresponding `.meta.json` file with recording parameters:

```
{
  "channels": 2,
  "device": "AA:BB:CC:DD:EE:FF",
  "emg_rate_hz": 2000,
  "file_index": 0,
  "opened_host_ms": 1738080622000,
  "opened_id": "20250128_143022",
  "proto_ver": null,
  "rotation_minutes": 30.0,
  "session_id": "20250128_143022",
  "started_host_ms": 1738080622000
}
```

Why Metadata Files?

- Self-documenting: no need to remember device config
- Enables batch processing: scripts can read FSR, sample rate, etc.
- Tracks rotation: `file_index` links segments of the same session

Metadata Update Policy:

- Written when file is opened
- Refreshed every 10 seconds (in case of crash)
- Final write when file is closed (rotation or program exit)

Rotation Logic

```
def rotate_if_needed(self):
    if (time.time() - self.t_open) >= self.rotate_sec:
        self.close()       # Flush and close current file
        self.idx += 1      # Increment rotation index
        self.open()        # Open new file with new idx
```

Rotation Triggers:

- Time-based: Every `--rotate-min` minutes (default 30)
- Manual: Ctrl+C stops recording, flushes all buffers

Rotation Guarantees:

- No data loss: all buffered rows are flushed before close
- Atomic writes: metadata written to `.tmp` file, then renamed (prevents corruption)
- Continuous timestamps: `dev_us` is monotonic across rotation boundaries

---

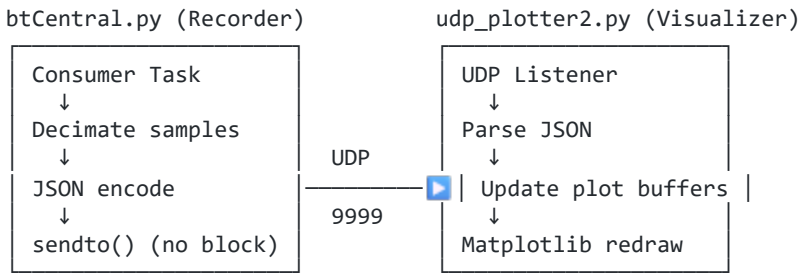
UDP Telemetry (Real-Time Plotting)

Purpose

Optional UDP stream for real-time visualization during recording. Useful for:

- Monitoring signal quality (EMG saturation, IMU clipping)
- Verifying sensor placement (live EMG waveform)
- Debugging protocol issues (frame gaps, CRC errors)

Architecture



UDP Packet Format (JSON)

All packets are JSON objects with a `"k"` (kind) field:

EMG Metadata

```
{
  "k": "emg_meta",
  "sr_hz": 2000,
  "ch": 2,
  "adc_bits": 12,
  "ymax": 4095,
  "seg": 0
}
```

Sent once when sample rate changes (or on reconnect).

RAW EMG Sample

```
{
  "k": "raw",
  "dev_us": 123456789,
  "t": 1738080622000,
  "raw": 2048,
  "seg": 0
}
```

Sent at `--udp-raw-hz` rate (default 500 Hz, decimated from 2000 Hz).

ENV EMG Sample

```
{
  "k": "env",
  "dev_us": 123456789,
  "t": 1738080622000,
  "env": 1024,
  "seg": 0
}
```

Sent at `--udp-env-hz` rate (default 50 Hz, decimated from 2000 Hz).

IMU Sample

```
{
  "k": "imu",
  "dev_us": 123456789,
  "t": 1738080622000,
  "amag": 1.05,
  "gmag": 12.3,
  "seg": 0
}
```

Sent at `--udp-imu-hz` rate (default 50 Hz, decimated from 200 Hz).
`amag` = accelerometer magnitude (g), `gmag` = gyroscope magnitude (dps).

Vitals

```
{
  "k": "vitals",
  "pct": 87,
  "v": 3.92,
  "spo2": 98,
  "hr": 72.5,
  "seg": 0
}
```

Sent every ~30 seconds when vitals frame is received.

Decimation Strategy

Why Decimate?

- RAW at 2000 Hz is too fast for UDP (packet loss, plotter lag)
- ENV at 2000 Hz is redundant (low-pass filtered, slow-varying)
- IMU at 200 Hz is overkill for magnitude plot

Decimation Ratios (Adaptive):

```
emg_raw_decim  = max(1, emg_rate_hz // udp_raw_hz)   # e.g., 2000 // 500 = 4
emg_env_decim  = max(1, emg_rate_hz // udp_env_hz)   # e.g., 2000 // 50 = 40
imu_decim      = max(1, imu_rate_hz // udp_imu_hz)   # e.g., 200 // 50 = 4
```

Decimation Implementation:

```
if (seq + i) % emg_raw_decim == 0:
    udp_send({"k": "raw", "raw": raw, ...})
```

Why Adaptive?

- Device sample rate can change (future feature)
- Plotter automatically adjusts timebase when `emg_meta` is received

UDP Best-Effort Policy

- **Non-blocking:** `sendto()` never blocks (socket is `setblocking(False)`)
- **No retries:** Dropped packets are acceptable (plotter interpolates gaps)
- **No ACKs:** One-way stream (recorder doesn't wait for plotter)

Why Best-Effort?

- Recording is the priority — UDP must never delay CSV writes
- Plotter is optional — recorder works standalone
- Packet loss is visible in plotter (gaps in waveform)

Watchdog & Reconnection

Problem Statement

BLE connections can stall due to:

- Radio interference (2.4 GHz congestion)
- ESP32 BLE stack bugs (rare but possible)
- Host Bluetooth adapter issues (Windows/Linux driver bugs)

Symptom: No BLE notifications for >5 seconds, but connection appears "up".

Watchdog Strategy

```
Watchdog Task (runs every 0.5 s)

age = now_ms() - last_notify_ms

if age > stall_reconnect_ms (default 5000 ms):
  1. Try resubscribe (stop_notify + start_notify)
  2. Wait 1 second
  3. If still stalled, trigger reconnect
```

Reconnection Flow

```
1. Watchdog detects stall (age > 5000 ms)
   ↓
2. Set reconnect_evt (signals main loop)
   ↓
3. Main loop exits BleakClient context
   ↓
4. Flush queue (discard stale data)
   ↓
5. Exponential backoff (0.5s, 1s, 2s, ..., max 20s)
   ↓
6. Reconnect to device
   ↓
7. Resubscribe to NUS_TX_UUID
   ↓
8. Send PING to kick device heartbeat
   ↓
9. Resume recording (no data loss after reconnect)
```

Reconnection Guarantees

- **No file corruption:** All buffered rows are flushed before reconnect
- **Continuous timestamps:** `dev_us` is monotonic (device clock never resets)
- **Gap detection:** `frame_seq` gaps are logged in health report
- **Automatic recovery:** No manual intervention required

Health Report During Reconnection

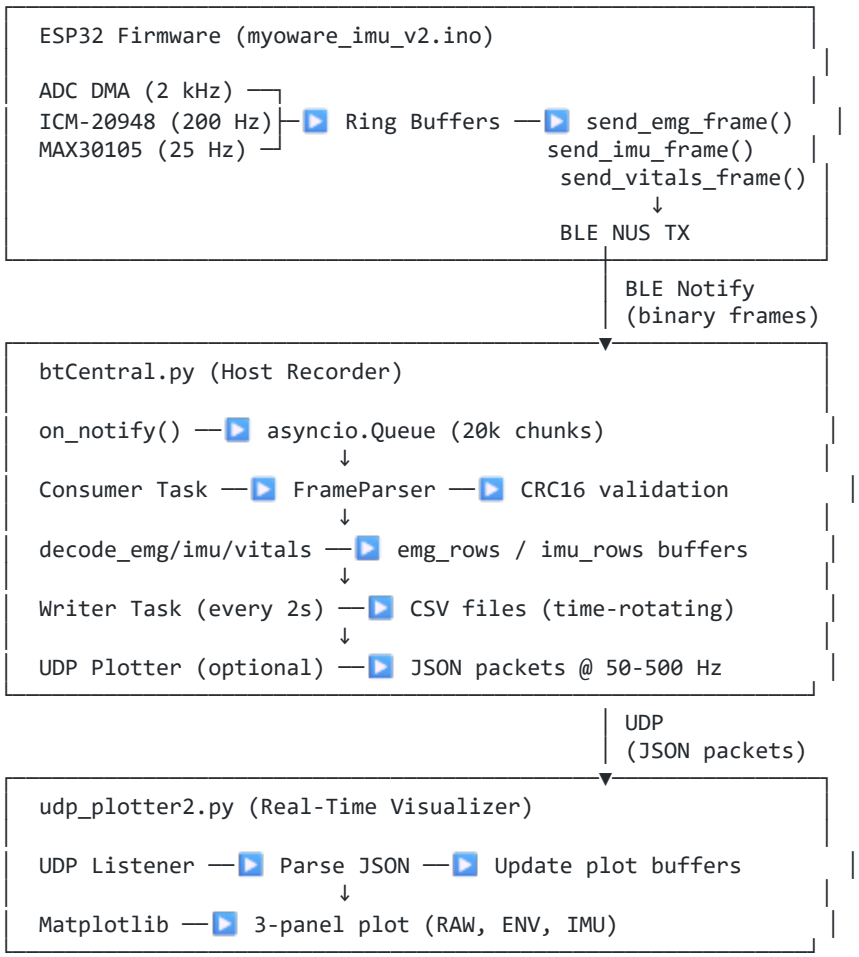
```
[Health] seg=0 off=05:23 state=STALL conn_sub=1 fps=0 notify/s=0 rx_kB/s=0.0 q=0
         crc=0 q_drop=0 imu_sat=0 batt=87% (3.92V)
         last_notify=5234ms last_good=5234ms ping_age=5234ms ping_fail=5
         gaps/s=0 recon=1 reason=stall_no_notify_5234ms
```

Key Fields:

- `state=STALL` — watchdog triggered
- `last_notify=5234ms` — time since last BLE notification
- `recon=1` — reconnection count
- `reason=stall_no_notify_5234ms` — why reconnect was triggered

Data Flow Diagram

End-to-End Data Path



Timing Diagram (Single Frame)

```
Time (ms)  ESP32                    BLE               Host
─────────  ─────                    ───               ────
0.0        ADC DMA fills buffer
           (64 samples @ 2 kHz)
           ↓
16.0       emgDrainTask wakes
           ↓
16.1       Back-calc timestamps
           ↓
16.2       emg_push() × 32 pairs
           ↓
16.5       send_emg_frame()
           ↓
16.6       BLE notify ──────────────▶
                                      ↓
                                      17.0  on_notify()
                                            ↓
                                      17.1  enqueue_chunk()
                                            ↓
                                      17.2  Consumer Task
                                            ↓
                                      17.3  FrameParser.feed()
                                            ↓
                                      17.4  CRC16 validation
                                            ↓
                                      17.5  decode_emg()
                                            ↓
                                      17.6  Append to emg_rows
                                            ↓
                                      19.0  Writer Task (every 2s)
                                            ↓
                                      19.1  CSV write
```

Latency Breakdown:

- ADC capture → DMA wakeup: ~16 ms (one DMA frame)
- DMA wakeup → BLE notify: ~0.5 ms (processing)
- BLE notify → Host receive: ~0.5 ms (BLE latency)
- Host receive → CSV write: ~2 s (buffered, flushed every 2 s)

Total latency (ADC → CSV): ~2.02 seconds (dominated by write buffering)

Usage Examples

Basic Recording (30 min rotation)

```
python btCentral.py --name MyoW1 --outdir logs --rotate-min 30
```

Output:

```
logs/
├── emg_log_20250128_143022_20250128_143022_000.csv
├── emg_log_20250128_143022_20250128_143022_000.meta.json
├── imu_log_20250128_143022_20250128_143022_000.csv
├── imu_log_20250128_143022_20250128_143022_000.meta.json
└── vitals_log.csv
```

Recording + Real-Time Plotting

Terminal 1 (Recorder):

```
python btCentral.py --name MyoW1 --outdir logs --rotate-min 30 \
  --udp-enable --udp-port 9999 \
  --udp-raw-hz 500 --udp-env-hz 50 --udp-imu-hz 50
```

Terminal 2 (Plotter):

```
python udp_plotter2.py --mode live --listen-port 9999
```

Long Session (2 hour recording, 20 min rotation)

```
python btCentral.py --name MyoW1 --outdir logs --rotate-min 20
```

Output (6 rotation segments):

```
logs/
├── emg_log_20250128_143022_20250128_143022_000.csv  ← 0-20 min
├── emg_log_20250128_143022_20250128_143022_001.csv  ← 20-40 min
├── emg_log_20250128_143022_20250128_143022_002.csv  ← 40-60 min
├── emg_log_20250128_143022_20250128_143022_003.csv  ← 60-80 min
├── emg_log_20250128_143022_20250128_143022_004.csv  ← 80-100 min
├── emg_log_20250128_143022_20250128_143022_005.csv  ← 100-120 min
├── (corresponding .meta.json files)
└── vitals_log.csv
```

Reconnection Stress Test

```
python btCentral.py --name MyoW1 --outdir logs \
  --stall-reconnect-ms 3000 \
  --max-backoff-sec 5.0
```

Aggressive reconnection:

- Reconnect after 3 seconds of silence (vs. default 5 s)
- Max backoff 5 seconds (vs. default 20 s)

Troubleshooting

High `q_drop` (Queue Drops)

Symptom:

```
[Health] ... q_drop=1234 ...
```

Causes:

- Host CPU overload (too many background processes)
- Slow disk I/O (USB 2.0 drive, network drive)
- Insufficient `--queue-max` (default 20,000)

Fixes:

- Close unnecessary programs
- Use local SSD (not network drive)
- Increase `--queue-max 50000`

High `imu_sat` (IMU Saturation)

Symptom:

```
[Health] ... imu_sat=567 ...
```

Causes:

- Accelerometer FSR too low (±2g) for high-impact motion
- Gyroscope FSR too low (±250 dps) for fast rotation

Fixes:

- Increase `IMU_ACC_FSR_G` in firmware (e.g., 4g or 8g)
- Increase `IMU_GYRO_FSR_DPS` in firmware (e.g., 500 dps or 1000 dps)

Frequent Reconnections

Symptom:

```
[Health] ... recon=12 reason=stall_no_notify_5234ms ...
```

Causes:

- BLE interference (2.4 GHz WiFi, microwave, etc.)
- Low device battery (<20%)
- Host Bluetooth adapter issues

Fixes:

- Move closer to device (reduce distance)
- Disable 2.4 GHz WiFi on host (use 5 GHz or Ethernet)
- Replace device battery
- Try different Bluetooth adapter (USB dongle)

CRC Errors

Symptom:

```
[Health] ... crc=45 ...
```

Causes:

- BLE packet corruption (rare)
- Firmware bug (very rare)

Fixes:

- Check firmware version (ensure latest)
- Reduce BLE distance
- If persistent, file a bug report with logs

Protocol Version History

Version	Date	Changes
`0x01`	2025-01	Initial protocol (no IMU FSR metadata)
`0x02`	2025-01	Added `FLAG_IMU_META_V2` (FSR codes in IMU frames), battery fields in vitals

Backward Compatibility:

- Host parser handles both v1 and v2 frames
- v1 frames: assume default FSR (±2g, ±250 dps)
- v2 frames: read FSR from `acc_fsr_code` / `gyro_fsr_code` fields

Future Enhancements

Planned Features

1. **Dynamic Sample Rate Adjustment**

- Host sends `SETRATE:<hz>` command
- Device adjusts EMG/IMU rates on-the-fly
- Useful for battery conservation during long sessions

1. **Compression**

- Delta encoding for EMG/IMU samples (reduce BLE bandwidth)
- LZ4 compression for CSV files (reduce disk usage)

1. **Cloud Upload**

- Auto-upload completed rotation segments to cloud storage
- Resume interrupted uploads on reconnect

1. **Multi-Device Recording**

- Record from multiple MyoWare devices simultaneously
- Synchronized timestamps via host clock

Known Limitations

1. **BLE Throughput**

- Max sustained rate: ~50 kB/s (limited by BLE 4.2 + MTU 247)
- 2 kHz EMG + 200 Hz IMU is near the limit
- Adding more sensors requires reducing sample rates

1. **Device Clock Wrapping**

- `frame_t_us` wraps every ~71.6 minutes
- Host extends to 64-bit, but requires continuous connection
- If disconnected >71 min, wrap detection may fail

1. **Vitals Latency**

- SpO2/HR computed every ~30 seconds (slow algorithm)
- Not suitable for real-time heart rate monitoring
- Use dedicated HR monitor for real-time feedback

References

- **Firmware:** `myoware_imu_v2.ino + CHANGELOG.md`
- **Host Recorder:** `btCentral.py`
- **Real-Time Plotter:** `udp_plotter2.py`
- **BLE Library:** Bleak (cross-platform BLE)
- **Protocol Spec:** This document (PROTOCOL.md)

Document Version: 1.0
Last Updated: 2025-01-28
Author: MyoWare IMU v2 Development Team